CO1-AT3-Coding Chellenge

View Only

Back

QUESTIONS

Q1
Smart University Payroll Management System
25 marks

Q2

Q1 Smart University Payroll Management System

25 marks

A university employs three categories of staff: Teaching Faculty, Technical Staff, and Visiting Faculty. The payroll system should calculate the monthly

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

KADIRI ASMITHA
192425038RD

teachingHours incentivePerHour Salary: basicSalary + (teachingHours × incentivePerHour) TechnicalStaff overtimeHours overtimeRate Salary: basicSalary + (overtimeHours × overtimeRate) VisitingFaculty lecturesHandled paymentPerLecture Salary: lecturesHandled × paymentPerLecture Override calculateSalary() in all subclasses. Create an array of Employee objects and demonstrate runtime polymorphism. Display: Employee details Monthly salary Highest-paid employee Average salary Search an employee using the employee ID.

Test Case 1 (Normal Case)

Input

3

TeachingFaculty
101
Dr. Arun
CSE
50000
20
500

TechnicalStaff
102
Mr. Ravi
ECE
35000
15
300

VisitingFaculty
103
Dr. Kumar
IT
0
40
1200

Search Employee ID:
102
Expected Output
Employee Details

Employee ID : 101
Name : Dr. Arun
Department : CSE
Monthly Salary : 60000.0

Employee ID : 102
Name : Mr. Ravi
Department : ECE
Monthly Salary : 39500.0

Employee ID : 103
Name : Dr. Kumar

Department : IT

Monthly Salary : 48000.0

Highest Paid Employee

Employee ID : 101

Salary : 60000.0

Average Salary

49166.67

Employee Found

Employee ID : 102

Name : Mr. Ravi

Salary : 39500.0

Your Code

```
import java.util.*;

class Main
{
    public static String readInput(Scanne
    {
        String s = "";

        while(sc.hasNextLine())
        {
            s = sc.nextLine().trim();

            if(!s.isEmpty())
            {
                return s;
            }
        }

        return s;
    }

    public static void main(String args[]
    {
        Scanner sc = new Scanner(System.i

        int n = Integer.parseInt(readInpu

        Employee emp[] = new Employee[n];

        for(int i=0;i<n;i++)
```

```
        {
            String type = readInput(sc);

            int id = Integer.parseInt(rea
            String name = readInput(sc);
            String department = readInput
            double basicSalary = Double.p

            if(type.equalsIgnoreCase("Tea
            {
                int teachingHours = Integ
                double incentivePerHour =

                emp[i] = new TeachingFacu
                        basicSalary,teach
            }
            else if(type.equalsIgnoreCase
            {
                int overtimeHours = Integ
                double overtimeRate = Dou

                emp[i] = new TechnicalSta
                        basicSalary,overt
            }
            else if(type.equalsIgnoreCase
            {
                int lecturesHandled = Int
                double paymentPerLecture

                emp[i] = new VisitingFacu
                        basicSalary,lectu
            }
        }

        String search = readInput(sc);

        if(search.equalsIgnoreCase("Searc
        {
            search = readInput(sc);
        }

        int searchId = Integer.parseInt(s

        Employee highest = emp[0];
        double totalSalary = 0;

        System.out.println("Employee Deta
        System.out.println();
```

```

        for(int i=0;i<n;i++)
        {
            emp[i].displayEmployee();

            System.out.println("Monthly S
            System.out.println();

            totalSalary += emp[i].calcula

            if(emp[i].calculateSalary() >
            {
                highest = emp[i];
            }
        }

        System.out.println("Highest Paid
        System.out.println("Employee ID :
        System.out.println("Salary : " +
        System.out.println();

        System.out.println("Average Salar

        double avg = totalSalary / n;

        if(avg == Math.floor(avg))
            System.out.println(avg);
        else
            System.out.printf("%.2f%n",av

        System.out.println();

        boolean found = false;

        for(int i=0;i<n;i++)
        {
            if(emp[i].getEmployeeId()==se
            {
                System.out.println("Emplc
                System.out.println("Emplc
                System.out.println("Name
                System.out.println("Salar

                found = true;
                break;
            }
        }

```

```
        if(!found)
        {
            System.out.println("Employee
        }
    }
}

class Employee
{
    private int employeeId;
    private String employeeName;
    private String department;
    private double basicSalary;

    public Employee()
    {

    }

    public Employee(int employeeId,String
                    String department,dou
    {
        this.employeeId=employeeId;
        this.employeeName=employeeName;
        this.department=department;
        this.basicSalary=basicSalary;
    }

    public int getEmployeeId()
    {
        return employeeId;
    }

    public String getEmployeeName()
    {
        return employeeName;
    }

    public String getDepartment()
    {
        return department;
    }

    public double getBasicSalary()
    {
        return basicSalary;
    }

```

```
    public double calculateSalary()
    {
        return basicSalary;
    }

    public void displayEmployee()
    {
        System.out.println("Employee ID :
        System.out.println("Name : " + em
        System.out.println("Department :
    }
}
class TeachingFaculty extends Employee
{
    private int teachingHours;
    private double incentivePerHour;

    public TeachingFaculty()
    {

    }

    public TeachingFaculty(int employeeId
                           String departm
                           int teachingHo
    {
        super(employeeId,employeeName,dep
        this.teachingHours = teachingHour
        this.incentivePerHour = incentive
    }

    @Override
    public double calculateSalary()
    {
        return getBasicSalary() + (teachi
    }
}

class TechnicalStaff extends Employee
{
    private int overtimeHours;
    private double overtimeRate;

    public TechnicalStaff()
    {

    }

```

```
    public TechnicalStaff(int employeeId,
                            String departme
                            int overtimeHou
    {
        super(employeeId,employeeName,dep
        this.overtimeHours = overtimeHour
        this.overtimeRate = overtimeRate;
    }

    @Override
    public double calculateSalary()
    {
        return getBasicSalary() + (overti
    }
}

class VisitingFaculty extends Employee
{
    private int lecturesHandled;
    private double paymentPerLecture;

    public VisitingFaculty()
    {

    }

    public VisitingFaculty(int employeeId
                             String departm
                             int lecturesHa
    {
        super(employeeId,employeeName,dep
        this.lecturesHandled = lecturesHa
        this.paymentPerLecture = paymentP
    }

    @Override
    public double calculateSalary()
    {
        return getBasicSalary() + (lectur
    }
}
```

Input

```
3

TeachingFaculty
101
```

Output

```
Employee Details

Employee ID : 101
Name : Dr.Arun
Department : CSE
Monthly Salary : 60000.0
```

Submitted